Cache Your Prompt When It's Green

Carbon-Aware Caching for Large Language Model Serving

Yuyang Tian, Desen Sun, Yi Ding, Sihang Liu
University of Waterloo Purdue University
SIGMETRICS 2026 PACM Meas. Anal. Comput. Syst.

Research meeting: 11 November 2026
Yu-Jui Liang Advisor: Hsiang-Yun Cheng
National Taiwan University / Academia Sinica

Source: GreenCache, title page

Discussion Index

Why Do We Need GreenCache?

Reusing KV from SSD saves prefill computation, but adds storage carbon.

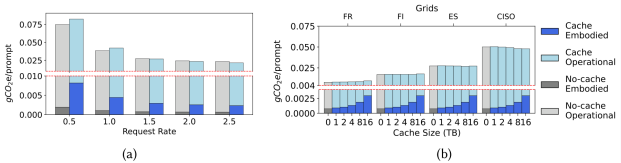


Fig. 7. Carbon emissions per request (a) in the ES grid under different request rates, and (b) under different cache sizes and grids (based on grid average CI in Figure 2a).

Embodied carbon Hardware manufacturing emissions, amortized over hardware allocation time.

Operational carbon Emissions from serving electricity: energy use $\times$ grid carbon intensity.

Trend in Fig. 7(a) Cache embodied carbon per prompt drops most visibly as request rate rises: the same allocated SSD cost is shared across more prompts.

Why it matters At low request rates, SSD embodied carbon outweighs operational savings. More requests dilute this cost; manufacturing emissions themselves do not decrease.

More KV caching is not always more carbon-efficient.

Source: GreenCache, Secs. 2.3, 3.2; Fig. 7

KV Reuse Table 1 Equations

Main Discussion index 2 / 48

Why Is a Fixed Cache Size Suboptimal?

Fixed 16 TB cache, 1.5 prompts/s. Carbon ratio = cache / no cache: **below 1** is a reduction; **Above 1** is an increase.

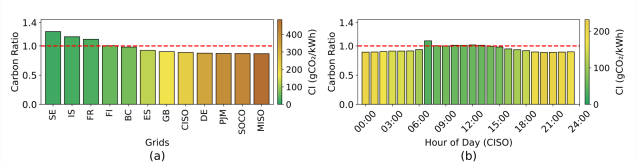


Fig. 8. Carbon emission savings from caching in 12 grids. A ratio < 1 indicates carbon emission reduction.

Across grids, Fig. 8(a)

Low CI, e.g. FR Cleaner electricity means less carbon avoided by skipping prefill. Added SSD embodied carbon can raise total emissions.

High CI, e.g. MISO The same energy saving avoids more carbon, enough to offset the SSD embodied cost.

Source: GreenCache, pp. 8–9, Fig. 8; Takeaway 5

Within a day, Fig. 8(b)

CISO's generation mix changes over time, so its CI changes too.

Low-CI hours Caching saves less carbon and may increase total emissions.

High-CI hours Avoiding repeated computation saves more carbon, making caching more beneficial.

Grid CI | Adapt Illustration

What Does GreenCache Optimize?

Choose allocated SSD capacity to minimize estimated serving carbon while meeting two latency requirements.

$$\min_{S \in \mathcal{S}} \quad C_{\text{operational}}(S) + C_{\text{embodied, allocated}}(S)$$

$$\text{subject to} \quad \text{TTFT attainment} \geq 90\%, \qquad \text{TPOT attainment} \geq 90\%.$$

Each metric allows at most 10% of requests to exceed its latency threshold.
TTFT and TPOT are evaluated separately.

Carbon accounting Operational carbon = energy × CI. Embodied carbon is amortized by hardware lifetime and usage time. GPU, CPU, and DRAM configurations are fixed; only SSD cache capacity changes dynamically.

Inputs and decision The solver uses predicted request rate, predicted CI, profiled performance/power, current performance feedback, and preset SLO constraints to select SSD cache capacity $S \in \mathcal{S}$.

Replacement policy LCS decides which KV cache entries to retain or evict within the selected capacity; the ILP decides the capacity.

Source: GreenCache, Secs. 3.2, 5.4–5.5, Eq. (6)

Embodied Carbon | Carbon Model

How Does GreenCache Work? (1/2): Monitoring & Prediction

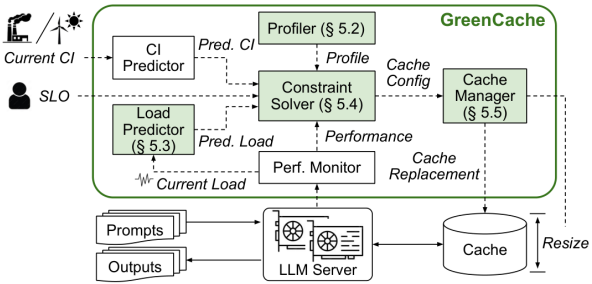


Fig. 10. System overview of GreenCache. Components in green are newly designed by this work.

Profiler
Profiles TTFT, TPOT, and power across cache sizes and request rates.

Perf. Monitor
Tracks current request rate, TTFT, and TPOT.

Load Predictor
Seasonal Autoregressive Integrated Moving Average (SARIMA) forecasts future request rates.

CI Predictor
Uses EnsembleCI from prior work to forecast grid carbon intensity.

Hourly resizing, 1 TB granularity. Serving continues during optimization.

Source: GreenCache, Secs. 4–5.3, Fig. 10

Profiles Predictors

How Does GreenCache Work? (2/2): Optimization & Execution

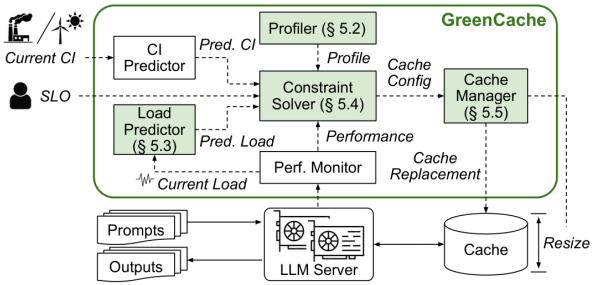


Fig. 10. System overview of GreenCache. Components in green are newly designed by this work.

SLO Input
Predefined TTFT / TPOT thresholds; $\geq$ 90% attainment each.

Constraint Solver
ILP selects the lowest-carbon feasible SSD capacity.

Cache Manager
Resizes SSD allocation and evicts entries using LCS.

LLM Server & Cache
LMCache/vLLM serves requests and reuses stored KV.

Hourly resizing, 1 TB granularity. Serving continues during optimization.

Source: GreenCache, Secs. 5.4–5.5, 6.1, 6.4; Fig. 10

Solver | Full ILP | SLO Numbers

How Does the Solver Decide? (1/2): Variables & Inputs

$$S^* = \arg\min_{S \in \mathcal{S}} \widehat{C}_{\text{total}}(S)$$

Conceptual summary. The full integer linear program is Equation (6).

1. Decision Variable — S Allocated SSD cache capacity; the only dynamically selected hardware capacity.

2. Candidate Set — $\mathcal{S}$ Discrete SSD cache sizes, with 1 TB allocation granularity.

3. Operational Carbon Estimated prefill energy multiplied by predicted grid carbon intensity.

4. Embodied Carbon Amortized carbon from allocated SSD capacity and fixed server hardware.

5. Model Inputs Predicted load/CI, profiled TTFT/TPOT and power, plus runtime performance feedback.

Source: GreenCache, Secs. 5.2–5.4, Eq. (6)

Variables Carbon Inputs Candidates

How Does the Solver Decide? (2/2): Constraints

$$S^* = \arg\min_{S \in \mathcal{S}} \widehat{C}_{\text{total}}(S)$$

Conceptual summary. The full integer linear program is Equation (6).

A. SLO Constraints — Equation (6)

1. TTFT Attainment At least 90% of requests meet the TTFT threshold.

2. TPOT Attainment At least 90% of requests meet the TPOT threshold.

B. Capacity Domain & Modeling Assumptions — § 5.4.2

Modeling assumptions used to build the search. Equation (6) lists the attainment sums above, and does not list this group as mathematical constraints.

3. Feasible Cache Sizes Discrete 1 TB steps, bounded by configured SSD capacity.

4. Decision Interval Hourly resizing; CI assumed constant within each interval.

5. Profile-Based Feasibility SLO feasibility estimated using warmed-up serving profiles.

Source: GreenCache, Sec. 5.4, Eq. (6), § 5.4.2

SLO Constraints | Assumptions | Full ILP | Profiles

How Does LCS Rank KV Cache Entries?

Solver chooses capacity → Cache Manager resizes → LCS ranks entries for eviction

Solver selects SSD capacity; Cache Manager allocates or releases it; LCS chooses entries to evict when reclaiming space.

$$\text{Score} = \frac{\#\text{Token} \times \#\text{Hit}}{\text{Size} \times \text{Age}} \qquad \text{Eq. (7)}$$

#Token

Accumulated Hit Tokens

Total tokens reused from this entry. More reuse means more avoided prefill computation.

#Hit

Cache Hit Count

Times this entry is reused. More hits suggest higher future reuse potential.

Size

Storage Footprint

SSD space occupied by the entry. Larger entries consume more storage capacity.

Age

Time in Storage

Time this entry has remained stored. Older entries receive a larger score penalty.

Lower LCS Score → Higher Eviction Priority

Source: GreenCache, Sec. 5.5, Eq. (7)

How Does LCS Work for Multi-Turn Conversations?

ShareGPT: task-specific Equation (8)

$$\text{Score}_{\text{chat}} = \frac{\text{CurTurn} \times \#\text{AccuToken}}{\text{Size} \times \text{Age}}$$

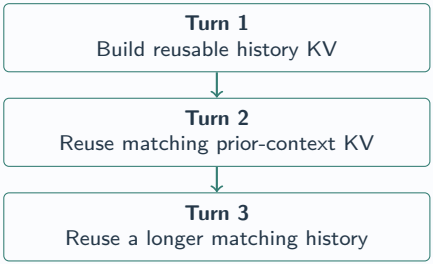
CurTurn

Current conversation turn index. Later turns typically rely on longer prior contexts; deeper conversations are favored, all else equal.

#AccuToken

Accumulated reused tokens across turns. More accumulated reuse means more avoided prefill computation.

History KV reuse across turns



Reuse requires matching, retained KV. New tokens still need computation; hits are not guaranteed.

Conversation-specific LCS prioritizes conversation depth and accumulated token reuse.

Source: GreenCache, Sec. 5.5, Eq. (8); Sec. 6.3.2

Discussion Index

How Does LCS Work for Document QA?

TriviaQA: task-specific Equation (9)

$$\text{Score}_{\text{doc}} = \frac{\#\text{Hit} \times \text{AccuDocLen}}{\text{Size} \times \text{Age}}$$

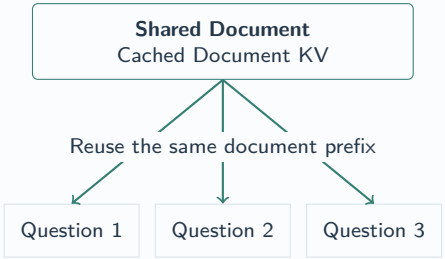
#Hit

Document cache hits across questions. Frequently reused documents receive higher priority.

AccuDocLen

Accumulated length of reused document content. More reused document context means more avoided prefill work.

One document, multiple questions



Matching retained document KV is reused; each question's new tokens still need computation.

Document-specific LCS prioritizes frequently reused documents and accumulated reused content.

Source: GreenCache, Sec. 5.5, Eq. (9); Secs. 6.1, 6.3.2

TriviaQA Workload / Zipf α

Why Use LCS Instead of FIFO or LRU?

Policy	How Does It Work?	Limitation / Potential Problem
FIFO	Evicts the earliest inserted cache entry.	Ignores reuse frequency and value; frequently reused entries may be evicted simply because they are old.
LRU	Evicts the least recently accessed entry.	Does not directly consider reused token volume, hit frequency, or storage footprint.
LCS (Ours)	Ranks entries by reused tokens, hit count, size, and age.	Uses a heuristic proxy rather than measuring actual CO_2 savings per entry.

Inferred from each mechanism; not a separately measured failure case.

Representative result. Llama-3 70B; TriviaQA, $\alpha = 0.7$; cache size 4 TB; metric: token-level cache hit rate.

FIFO: 23% **LRU: 26%** **LCS: 35%**

LCS improves token-level cache hit rate by 9 percentage points over LRU.

LCS is especially useful when cache capacity is limited.

Token-level hit rate = reused input tokens / total input tokens.

Source: GreenCache, Section 6.3.2, Table 3.

Full Table 3

Real Prototype or Simulator?

A. Real GPU Prototype

Built on **LMCache + vLLM** with continuous batching, using NVIDIA L40 GPUs.

B. Trace-Driven Evaluation

ShareGPT / TriviaQA workloads, Azure-based request arrivals, and CarbonCast carbon intensity.

C. Power Measurement & Carbon Modeling

CPU/GPU power measured. SSD/DRAM power from specifications; embodied carbon modeled and amortized.

Key Takeaway

No standalone GPU-serving simulator is reported. GreenCache combines real serving measurements, trace-driven workloads, and carbon models.

Source: GreenCache, Secs. 5.2–5.4, 6.1

Measurement Notes

How Is GreenCache Evaluated?

A. Models & Workloads

Llama-3 8B / 70B with ShareGPT and TriviaQA workloads.

B. Three Strategies Compared

No Cache	Full Cache	GreenCache
No context caching	Maximum configured capacity	Adaptive capacity + LCS

Full Cache and GreenCache share the same maximum capacity.

C. Evaluation Metrics

Total carbon per prompt · TTFT / TPOT SLO attainment · Allocated cache capacity

TTFT and TPOT each require 90% attainment, separately.

Key Question

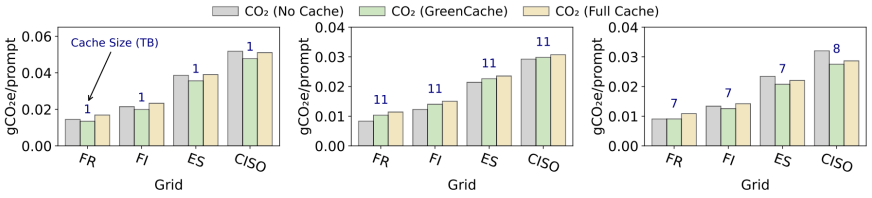
Can adaptive cache sizing reduce total carbon while satisfying serving latency SLOs?

Source: GreenCache, Sec. 6.1

SLO Thresholds

Q1: Does GreenCache Reduce Carbon?

Llama-3 70B across four grids. Lower bars mean lower total carbon per prompt; blue labels show cache size (TB).



(a) Carbon emissions of Llama-3 70B.

Fig. 12. Average carbon emissions of LLM tasks.

Adaptive sizing saves carbon versus Full Cache by avoiding unnecessary cache capacity.

Prefix reuse helps offset cache costs, but more reuse does not guarantee larger carbon savings.

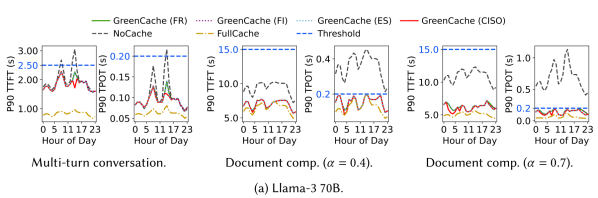
No Cache is lower for $\alpha = 0.4$, but it must also meet latency SLOs. The next slide checks this tradeoff.

Source: GreenCache, Sec. 6.2, Fig. 12a

Llama-3 8B

Q2: Does GreenCache Still Meet SLOs?

Llama-3 70B. P90 below the dashed threshold means at least 90% attainment, checked separately for TTFT and TPOT.



(a) Llama-3 70B.

Fig. 13. SLO attainment timelines of LLM tasks.

No Cache

Often exceeds latency SLOs. Lower carbon alone does not make it a feasible alternative.

GreenCache

Slightly higher latency than Full Cache, but still meets the SLOs in these experiments.

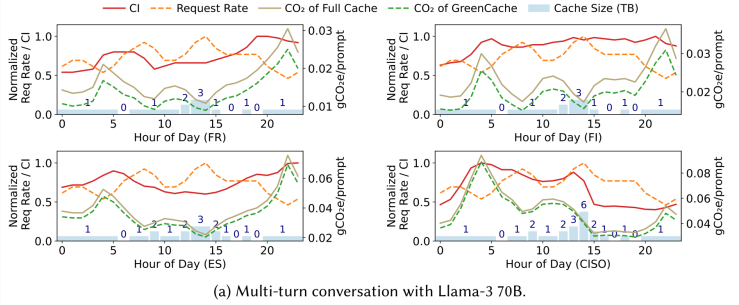
GreenCache uses Full Cache's latency headroom to reduce cache capacity and total carbon while keeping SLOs satisfied.

Source: GreenCache, Secs. 6.1–6.2, Fig. 13a

8B Timelines

Q3: Does Capacity Track Load and CI?

Llama-3 70B, multi-turn. Normalized CI and request rate, carbon per prompt, and the chosen cache size.



(a) Multi-turn conversation with Llama-3 70B.

Fig. 14. Timelines of carbon emissions under variable CI and rate.

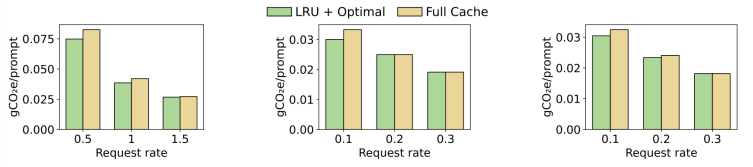
Adaptive capacity generally lowers carbon per prompt versus Full Cache in these timelines.
Higher load can require more cache to meet SLOs; higher CI makes avoided computation more valuable.
Savings depend on the joint tradeoff, not simply on higher CI or a higher request rate.

Source: GreenCache, Sec. 6.2, Fig. 14a

Document Timeline

Q4: Does Adaptive Sizing Helps?

Ablation: LMCache's LRU plus GreenCache's adaptive capacity ("LRU + Optimal"), versus Full Cache. ES grid average CI, 124 gCO_2e/kWh. Llama-3 70B.



(a) Multi-turn conversation. (b) Document comp. (α=0.4). (c) Document comp. (α=0.7).

Fig. 15. Ablation study on adaptive caching (Llama-3 70B).

Even without LCS, GreenCache's adaptive capacity decision still reduces carbon versus Full Cache. In this experiment, the effect is clearer at low load, because the cache needed to meet the SLO is smaller.

Capacity adjustment itself contributes. This does not mean LCS has no additional value.

Source: GreenCache, Sec. 6.3.1, Fig. 15

Profiling Heatmaps

Conclusion

1. Problem

Context caching trades avoided prefill for provisioned SSD. Total carbon includes both terms.

2. Contribution

Profiling, load and CI forecasts, an ILP capacity choice, and LCS, under separate TTFT and TPOT attainments.

3. Evidence

Versus Full Cache, studied settings cut carbon while P90 stays inside the authors' thresholds. 15.1% / 25.3% is FR, Llama-3 70B, multi-turn.

4. Future work

Consider data centers in different regions, and HBM–HBF as cache tiers under the same SLO and embodied-carbon tradeoff. Our plan; GreenCache does not evaluate either.

Source: GreenCache, Secs. 6–7. Future work is ours, not an author result.

Limits and HBF

Discussion Index

Main answers the talk. This index opens the evidence behind a question.

Topic	What is here	Jump
A. Motivation	KV reuse, grids, latency, embodied carbon, Equations (1)–(5)	Open A
B. Method	Solver variables, Equation (6), assumptions, LCS	Open B
C. Evidence	Measurement boundary, 8B results, document timeline, solver time	Open C
D. Sensitivity	Forecast error, resize interval, SSD life, SSD carbon, limits	Open D

Is it a simulator? M13 Notes
System walkthrough 1/2 2/2
Solver decision 1/2 2/2
ILP, SLOs, domain Eq. 6 SLOs Domain

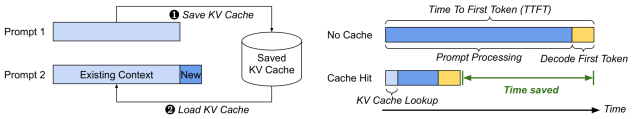
Back to Main 19

8B and the other task Fig. 12 Fig. 14b
LCS: General Chat Doc QA Policies Table 3
LCS examples: Multi-turn Document QA
TriviaQA / Zipf Workload
What we would not claim Limits

Discussion Index

Why Cache Repeated Context?

Multi-turn chats and document questions resend a shared context. A hit reloads KV instead of recomputing that prefill.



(a) Caching procedure. (b) Timeline of no cache and a cache hit.

Fig. 1. Illustration of caching for LLM serving.

What a hit saves Existing context is loaded from storage and stitched with the new tokens. Prefill of the repeated context is skipped.

What a hit does not save Autoregressive decode FLOPs remain. Decode latency can still fall because continuous batching waits less.

Source: GreenCache, Sec. 2.2, Fig. 1

ShareGPT, Llama-3 70B, 4× L40 Average prompt-processing TTFT is 1.7 s. Loading the previous context's KV averages 0.03 s.

How to read those two times They are not the same work item, and they are not an end-to-end speedup of one request.

Latency Detail
Context Length
Back to Main 2
Discussion Index

Which Grids, and How Fast Does CI Move?

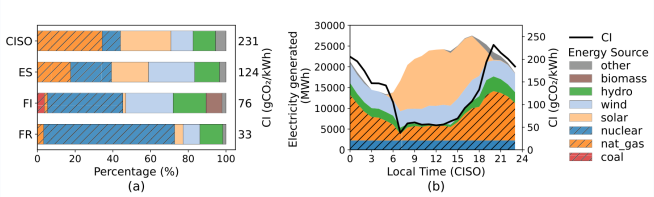


Fig. 2. (a) Average carbon intensity (CI) and energy sources of four grids in 2024 [18]. (b) CI variation due to energy sources of the CISO grid on July 6, 2022 [49].

2024 averages in Fig. 2a FR 33, FI 76, ES 124, and CISO 231 gCO_2e/kWh. The mix, not a single "green grid" label, sets operational carbon.

One CISO day Fig. 2b is 6 July 2022. Generation and CI move together. GreenCache's hourly decision uses this kind of variation, via CarbonCast and EnsembleCI.

Crop note The supplied Fig. 2 PNG has a few orphan pixels from a cut-off title. This slide drops the top 16 px. Axes, CI labels, and the legend are unchanged.

Source: GreenCache, Sec. 2.3, Fig. 2

Back to Main 3 Next

Discussion Index

Where Does Caching Change Latency?

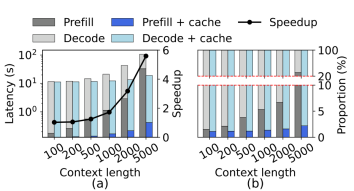


Fig. 3. (a) Latency and speedup from caching under different context lengths. (b) Latency breakdown.

Longer cached context, larger prefill speedup Fig. 3a. The prefill bar shrinks more than the decode bar as context grows.

Why decode moves at all Fig. 3b: caching does not remove decode computation. It shortens the wait in continuous batching.

Setup for this section Llama-3 70B, ShareGPT, the Table 1 server, LMCache on 16 TB, cache warmed with 200k prompts, 500 prompts recorded.

Source: GreenCache, Sec. 3.1.1, Fig. 3

Back to Main 2 | KV Reuse | Next | Discussion Index

How Long Is the Repeated Context?

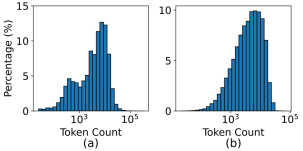


Fig. 4. Distribution of context length in (a) ShareGPT [30] and (b) TriviaQA [32].

ShareGPT 77.2% of prompts carry more than 1000 tokens from earlier turns. Length, and therefore the value of a hit, varies inside the same task.

TriviaQA The reused context is the document. Average document length in the paper's discussion is 5880 tokens.

Design consequence Takeaway 1: longer contexts should be preferred when the cache cannot hold everything. LCS later puts token volume in the score.

Source: GreenCache, Sec. 3.1.1, Fig. 4

Back to Main 2 KV Reuse Next

Discussion Index

What Does Request Rate Change?

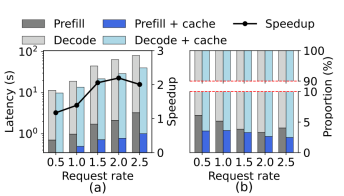


Fig. 5. (a) Latency of prefill and decode under different request rates, and the speedup from caching. (b) Fraction of prefill and decode latency.

Higher rate, more latency reduction Both prefill and decode improve more as the offered rate rises, because a shorter prefill also frees the batch.

Carbon follows the same direction Takeaway 2 and Fig. 7a: at low rate the SSD's embodied carbon can dominate; at high rate the avoided energy dominates.

SLO coupling A busy hour may need a large cache to keep attainment, even if that hour's CI is low.

Source: GreenCache, Sec. 3.1.2, Fig. 5

Back to Main 2 Next

Discussion Index

Does a Bigger Cache Always Raise the Hit Rate?

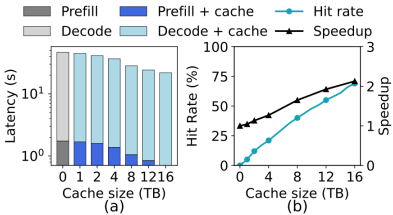


Fig. 6. (a) Latency and speedup from caching under different cache sizes. (b) Cache hit rate.

Fixed load 1.5 prompts/s, cache from 1 to 16 TB, compared with no cache.

Diminishing hits Latency and speedup improve with size, then the hit-rate gain slows. Takeaway 3: larger is better for latency, not linearly, and not automatically for carbon.

Hand-off to carbon Fig. 7b then shows the embodied share rising with those same sizes.

Source: GreenCache, Sec. 3.1.3, Fig. 6

Back to Main 3 Next

Discussion Index

Table 1: Embodied Carbon on This Server

Component	Type	Embodied carbon
CPU	AMD 7453	9.3 $kgCO_2e$
GPU	4× NVIDIA L40	106.4 $kgCO_2e$
Memory	512 GB DDR4	30.8 $kgCO_2e$
Storage	up to 16 TB SSD	up to 480 $kgCO_2e$

76.6% The paper states that SSD embodied carbon is 76.6% of this server's embodied carbon at the Table 1 configuration. $480/(9.3 + 106.4 + 30.8 + 480) = 76.6\%$.

Scope Prior studies are cited for SSDs exceeding 75% of datacenter-server embodied carbon. That citation is not a measurement of every server.

Accounting choice Only the allocated SSD capacity is charged, for the fraction of lifetime it is reserved. Default lifetime is 5 years unless a sensitivity slide says otherwise.

Source: GreenCache, Sec. 2.3, Table 1. Sum checked against the stated 76.6%.

Back to Main 4 Main 2 Next Discussion Index

Carbon Accounting, Equations (1)–(5)

$$C = C_o + \frac{T}{\mathrm{LT}} C_e \qquad C_o = E \times \mathrm{CI}$$

$$C_e = C_{e,\mathrm{GPU}} + C_{e,\mathrm{CPU}} + C_{e,\mathrm{Mem}} + C_{e,\mathrm{SSD}}$$

$$C_{e,\mathrm{Cache}} = S_{\mathrm{Alloc}} \times \frac{T}{\mathrm{LT}} \times C_{e,\mathrm{SSD}}^{\mathrm{Unit}}$$

$$C = E \times \mathrm{CI} + \frac{T}{\mathrm{LT}} \left(C_{e,\mathrm{Cache}} + C_{e,\mathrm{Others}} \right)$$

Two different objects Per-prompt carbon in the plots is not the same number as the SSD-only embodied term, and neither is "GPU energy saved."

What caching changes It can lower E and T. It raises S_{Alloc}. Equation (5) is the tradeoff GreenCache searches.

Unit carbon Table 1's 480 $\mathrm{kgCO_2e}$ over 16 TB is 30 $\mathrm{kgCO_2e/TB}$. Fig. 20 then varies that assumption from 30 to 90.

Source: GreenCache, Secs. 2.3, 3.2.1, Eqs. (1)–(5)

Back to Main 4 Main 2 Solver Next Discussion Index

Load and CI Push Capacity in Opposite Ways

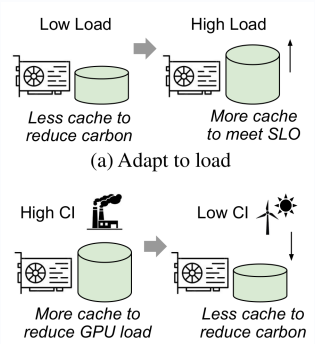


Fig. 9. Adaptive caching.

Source: GreenCache, Sec. 4.1, Fig. 9

High load Fig. 9a grows the cache. The stated reason is SLO attainment, not carbon alone.

Low CI Fig. 9b shrinks the cache to avoid SSD embodied carbon. This is the forecast case, not a rule that low CI always means zero cache.

Why a forecast Growing the cache needs warm-up. Decisions look up to 24 hours ahead so the new capacity is warm when the load arrives.

Still an illustration The measured response is Fig. 14, not this cartoon.

Back to Main 3 Next

Discussion Index

What Do the Profiles Actually Contain?

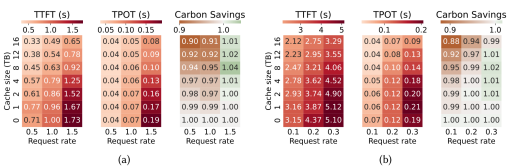


Fig. 11. Profiling results of TTFT and TPOT (lower is better), and carbon savings over no-cache (higher is better) when serving Llama-3 70B in the ES grid for (a) multi-turn conversation, using a ShareGPT dataset [30] and (b) document comprehension using a TriviaQA dataset [32] (with skewness of $\alpha = 0.4$).

Axes Llama-3 70B, ES grid. (a) ShareGPT multi-turn. (b) TriviaQA, $\alpha = 0.4$. Rows are cache size; columns are request rate.

Read the colors carefully TTFT and TPOT: lower is better. The carbon-savings panel is higher-better. The paper defines that ratio as no-cache over cached, so a value **above 1 is a benefit**. This is the opposite direction from Fig. 8, where a ratio below 1 is a reduction.

The pattern used by the solver Large caches help latency, especially at higher rate. At low rate a large cache can sit below 1 on the savings panel.

Source: GreenCache, Sec. 5.2, Fig. 11

Back to Main 5 | Main 18 | Solver | Next | Discussion Index

Full ILP and Variables

$$\underset{S_i^{j_t}}{\operatorname{argmin}} \sum_{i=1}^{N} \Big(p_i^{j_t}\, \mathrm{TTFT}_i^{j_t}\, \mathrm{CI}_t + \frac{\mathrm{TTFT}_i^{j_t}}{\mathrm{LT}_{\mathrm{SSD}}}\, S_i^{j_t}\, C_{e,\mathrm{SSD}}^{\mathrm{Unit}} + \sum_{\mathrm{comp}\in\{\mathrm{GPU},\mathrm{CPU},\mathrm{Mem}\}} \frac{t_{\mathrm{TTFT},i}^{j_t}}{\mathrm{LT}_{\mathrm{comp}}}\, C_{e,\mathrm{comp}} \Big)$$

$$\text{s.t.} \quad \sum_{i=1}^{N} z_{\mathrm{TTFT},i} \geq \rho N \;\wedge\; \sum_{i=1}^{N} z_{\mathrm{TPOT},i} \geq \rho N$$

Operational carbon. $p_i^{j_t}\,\mathrm{TTFT}_i^{j_t}\,\mathrm{CI}_t$ is estimated prefill energy times predicted intensity. **SSD embodied carbon.** Allocated $S_i^{j_t}$ times unit SSD carbon, amortized on the SSD lifetime.
Other hardware embodied carbon. GPU, CPU, and memory on their lifetimes. Eq. (6) prints $t_{\mathrm{TTFT},i}^{j_t}$.

Symbol	Table 2 meaning	Symbol	Table 2 meaning
t	Timestamp of execution	$p_i^{j_t}$	Average power of request i
CI_t	Carbon intensity at t	$\mathrm{TTFT}_i^{j_t}$	TTFT of request i
j_t	Request rate at t	$S_i^{j_t}$	Cache size of request i
N	Requests in the service	$\mathrm{LT}_{\mathrm{comp}}$	Lifetime of SSD, GPU, CPU, or Mem

CI: gCO_2e/kWh. TTFT: seconds. Allocation: TB. Lifetime: years. Power has no unit in Table 2. The next sentence sets $\rho = 0.9$.
Objective. Prefill carbon; the TPOT SLO carries decode latency. **Solver.** PuLP + COIN-OR CBC, average 7.03 s.

Source: GreenCache, Sec. 5.4, Table 2, Eq. (6), Sec. 6.4

Back to Main 7 | Arch. | Constraints | SLOs | Time | Discussion Index

Exact SLO Constraints

Equation (6) gives each latency metric its own binary indicator. The paper sets $\rho = 0.9$.

$$z_{\mathrm{TTFT},i} \in \{0,1\},\ z_{\mathrm{TTFT},i} = 1 \text{ if request } i \text{ meets the TTFT constraint, } \sum_{i=1}^{N} z_{\mathrm{TTFT},i} \geq \rho N.$$

$$z_{\mathrm{TPOT},i} \in \{0,1\},\ z_{\mathrm{TPOT},i} = 1 \text{ if request } i \text{ meets the TPOT constraint, } \sum_{i=1}^{N} z_{\mathrm{TPOT},i} \geq \rho N.$$

Two attainments. Each sum requires at least 90% of requests. The same requests need not clear both thresholds: a request can meet TTFT and miss TPOT. The paper's "both" names these two requirements.

Task	Model	TTFT SLO	TPOT SLO
Multi-turn	Llama-3 70B	2.5 s	0.2 s
Multi-turn	Llama-3 8B	0.5 s	0.15 s
Document	Llama-3 70B	15 s	0.2 s line in Fig. 13
Document	Llama-3 8B	see note	0.15 s line in Fig. 13

Unresolved 8B document TTFT. Section 6.1 relaxes that threshold to 2.5 s. The dashed line on that Fig. 13 panel is drawn at 4.0 s. Both are shown; this deck does not pick one. A P90 curve under the drawn threshold is the paper's picture of at least 90% attainment.

Source: GreenCache, Sec. 5.4, Eq. (6), Sec. 6.1, Fig. 13

Back to Main 8 | Arch. | Settings | Next | Discussion Index

Load Forecast and Carbon-Intensity Forecast

Load SARIMA (Seasonal Autoregressive Integrated Moving Average) captures daily periodicity and short-term autocorrelation. pmdarima fits parameters by maximum likelihood and auto-tunes them. Hold-out uses the latest three days to predict 24 hours; runtime updates hourly on separate CPU cores.

Carbon intensity EnsembleCI, an existing predictor, trained on 18 months of CarbonCast and evaluated on 6 July 2022. GreenCache does not claim to have designed EnsembleCI.

Errors, Sec. 6.5 Load MAPE 4.3%. CI MAPE: FR 12.7%, FI 15.3%, ES 11.3%, CISO 6.8%. Median context-length gap versus the profile: 5.78% (multi-turn) and 1.13% (document).

Effect on the carbon result Relative to an ideal run with ground truth, CI error trims 0.0064% of the savings on average, adding load error trims 0.20%, and adding profiler mismatch trims 0.79%.

Scope of optimality Best profiled candidate under forecasts and SLOs; forecast and profile errors can change the choice.

Source: GreenCache, Secs. 5.3, 5.4, 6.1, 6.5

Back to Main 5 | Solver | Next | Discussion Index

Detailed Multi-turn LCS Example

$$\text{Score}_{\text{chat}} = \frac{\text{CurTurn} \times \#\text{AccuToken}}{\text{Size} \times \text{Age}} \qquad \text{Eq. (8)}$$

CurTurn: Current turn index; deeper conversations tend to reuse longer history.
#AccuToken: Total tokens reused across turns, indicating avoided prefill work.
Size: SSD footprint. **Age:** Time stored, not time since the last hit.

Hypothetical entries — illustrative values, not paper measurements

Entry	CurTurn	#AccuToken	Size (GiB)	Age (h)	Score
A	2	200	1	2	$2 \times 200/(1 \times 2) = 200$
B	4	200	1	2	$4 \times 200/(1 \times 2) = 400$
C	4	600	2	2	$4 \times 600/(2 \times 2) = 600$

A vs. B: Doubling turn depth doubles the score, all else equal.
B vs. C: Three times the accumulated reuse outweighs twice the storage cost.
Evict A first: It has the lowest score. Deeper history and greater reuse favor retention.
This is a heuristic, not measured CO_2. Use consistent units; the example does not predict a real workload's hit rate.

Source: GreenCache, Sec. 5.5, Eq. (8). Numerical example is illustrative.

Back to Main 10 | General LCS | Document QA | Policies | Discussion Index

What Is Measured, Replayed, and Modeled?

Serving path LMCache + vLLM + continuous batching, plus the authors' LCS policy. No paper statement says they built or modified a standalone GPU simulator.

Requests Profiling prompts and evaluation prompts are disjoint. ShareGPT cache warm-up: 200k prompts. TriviaQA: 50k. The profiler then runs 500 prompts. Zipf $\alpha = 0.4$: about 10% of documents get about 25% of prompts. $\alpha = 0.7$: about 10% get about 50%.

Time and grid Azure LLM inference trace, downscaled so the peak fits this server's sustainable throughput. Main carbon run uses variable CI on 6 July 2022. Ablations and SSD sensitivities use the ES average, 124 gCO_2e/kWh, and fixed rates of 1.5 prompts/s (multi-turn) or 0.2 (document) where Sec. 6.6 says so.

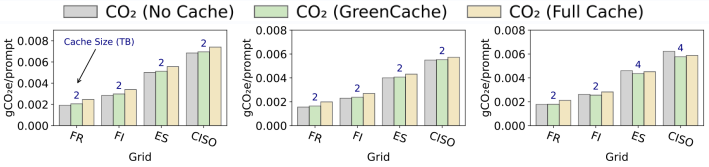
Power CPU: RAPL. GPU: pyNVML at 1 ms. SSD and DRAM: specification power, because those parts expose no direct measurement interface in the paper.

Context limit Both models use an 8k-token context window; extra context is truncated.

Source: GreenCache, Secs. 3, 5.2, 6.1, 6.6

Back to Main 13 | Evaluation | Next | Discussion Index

Figure 12b: Llama-3 8B Carbon



(b) Carbon emissions of Llama-3 8B.

Fig. 12. Average carbon emissions of LLM tasks.

Same three workloads, smaller machine Versus Full Cache the paper reports 10.8%, 7.6%, and 9.7% lower carbon for multi-turn, $\alpha = 0.4$, and $\alpha = 0.7$.

Do not stack this on the 70B bars 8B uses 2× L40, BF16, and an 8 TB maximum. 70B uses 4× L40, INT8, and 16 TB.

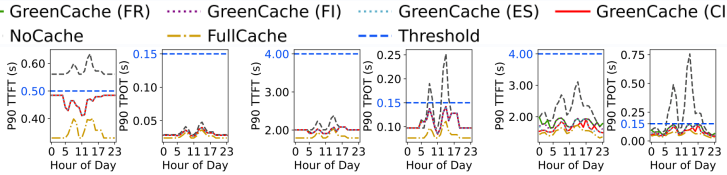
Legend The strip above is the original Fig. 12 legend: gray No Cache, green GreenCache, tan Full Cache. Blue numbers are cache size in TB.

Source: GreenCache, Sec. 6.2, Fig. 12b. Legend viewport is the original top strip.

Back to Main 15 Next

Discussion Index

Figure 13b: Llama-3 8B SLO Timelines



(b) Llama-3 8B.

Fig. 13. SLO attainment timelines of LLM tasks.

Multi-turn lines match the text TTFT threshold 0.5 s and TPOT threshold 0.15 s. GreenCache stays under them; No Cache crosses TTFT.

Document TTFT is unresolved The text sets 2.5 s. The dashed line in these document panels is at 4.0 s. TPOT in the figure is drawn at 0.15 s.

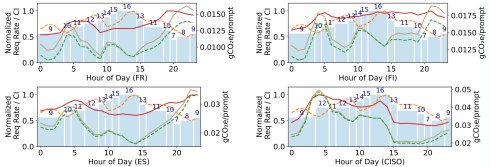
Reading P90 Staying under the dashed line is the paper's $\geq 90\%$ attainment test. The 70B half of this figure is on the main slide.

Source: GreenCache, Secs. 6.1–6.2, Fig. 13b

Back to Main 16 Next

Discussion Index

Figure 14b: Document Timeline, $\alpha = 0.4$



(b) Document comprehension ($\alpha = 0.4$) with Llama-3 70B.

Fig. 14. Timelines of carbon emissions under variable CI and rate.

Larger footprints Document context keeps the cache near the top of the 16 TB range for much of the day. The paper's multi-turn range versus Full Cache is 6.9–20.4%; the document range is 3.2–16.2%.

Less room to shrink When the SLO already needs a large cache, embodied-carbon savings get smaller. That is why this task saves less than multi-turn at the same hours of high load.

Legend Red CI, dashed orange request rate, brown Full Cache CO_2, dashed green GreenCache CO_2, blue bars cache size in TB. The FR 15.1% / 25.3% numbers belong to multi-turn, Fig. 14a, not to this panel.

Source: GreenCache, Sec. 6.2, Fig. 14

Back to Main 17 Next

Discussion Index

Solver Time per Resize

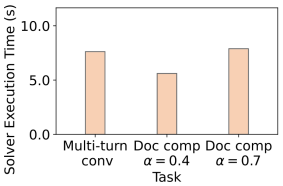


Fig. 16. Constraint solver execution time for every cache resize.

Cost of a decision Average 7.03 s to choose the next cache size, across the three tasks in Fig. 16.

Why that is acceptable in the paper The default resize interval is one hour, and the workloads are long-running. 7.03 s is overhead of the controller, not of the token path.

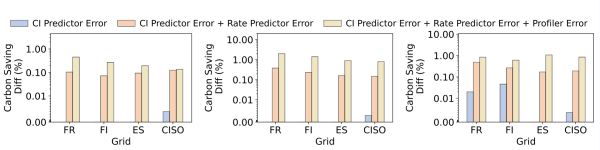
Not free in a faster loop Fig. 18 shows that resizing much faster than the hourly CI trace does not buy much carbon, while much slower resizing gives savings back.

Source: GreenCache, Sec. 6.4, Fig. 16

Back to Main 6 Next

Discussion Index

Do Forecast and Profile Errors Spend the Savings?



(a) Multi-turn conversation. (b) Document comp. ($\alpha = 0.4$). (c) Document comp. ($\alpha = 0.7$).

Fig. 17. Impact of prediction and profile inaccuracies (Llama-3 70B).

What the bars add Left to right within the paper's comparison: CI error, then CI plus load error, then those plus profiler mismatch. The vertical axis is the loss of carbon savings versus an ideal ground-truth run, on a log scale so the small values are visible.

Magnitudes Averages reported in Sec. 6.5: 0.0064%, 0.20%, and 0.79%. Load error moves the chosen size, so it matters more than its MAPE suggests. Profiler error weighs more on FR and FI, where embodied carbon is a larger share.

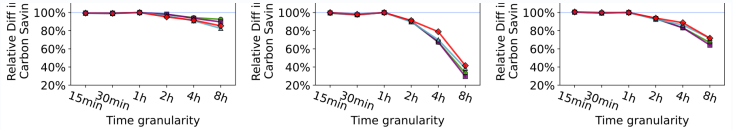
Limit of the claim "Low impact" is this comparison on Llama-3 70B and these four grids. It is not a bound for every forecast model.

Source: GreenCache, Sec. 6.5, Fig. 17

Back to Main 6 Next

Discussion Index

How Often Should the Cache Resize?



(a) Multi-turn conversation. (b) Document comp. ($\alpha = 0.4$). (c) Document comp. ($\alpha = 0.7$).

Fig. 18. Impact of variable cache resizing intervals (Llama-3 70B). A higher value indicates more savings.

Relative to the 1-hour default Fig. 18 plots carbon savings against that default. A higher value means more of the default saving remains. Intervals of 15 min, 30 min, 1 h, 2 h, 4 h, and 8 h are shown for all four grids.

Long intervals The solver must pick a size that protects the SLO for the whole interval, so it holds a larger cache and gives back savings. The drop is steepest for document QA at $\alpha = 0.4$, the case that already wants a large cache.

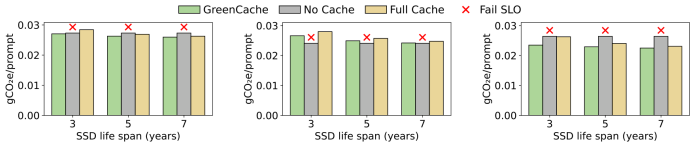
Shorter than one hour CarbonCast CI is hourly. Sub-hour decisions reuse that hour's CI. They stay close to the 1-hour result.

Source: GreenCache, Sec. 6.6.1, Fig. 18

Back to Main 6 Next

Discussion Index

SSD Lifetime Changes the Embodied Weight



(a) Multi-turn conversation. (b) Document comp. ($\alpha = 0.4$). (c) Document comp. ($\alpha = 0.7$).

Fig. 19. Variable SSD lifespan (Llama-3 70B, ES grid).

Setup Llama-3 70B, ES average CI, lifetimes 3, 5, and 7 years. Fixed rates: 1.5 prompts/s multi-turn, 0.2 prompts/s document. Default elsewhere in the paper is about 5 years.

Direction A 3-year life raises amortized SSD carbon, so shrinking the cache is worth more: the paper reports up to 11.9% savings in this study. A longer life makes the same resize less valuable.

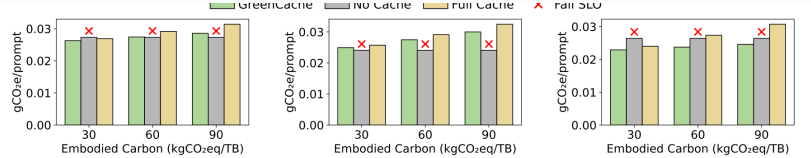
Markers The figure includes No Cache and a Fail-SLO marker. A low bar that fails the SLO is not counted as the GreenCache result.

Source: GreenCache, Sec. 6.6.2, Fig. 19

Back to Main 19 Next

Discussion Index

SSD Embodied Carbon Is an Assumption



(a) Multi-turn conversation. (b) Document comp. ($\alpha = 0.4$). (c) Document comp. ($\alpha = 0.7$).

Fig. 20. Variable SSD embodied carbon (Llama-3 70B, ES grid).

Range 30 to 90 $kgCO_2e/TB$, same rates and ES CI as the lifetime study. Table 1 corresponds to the 30 $kgCO_2e/TB$ end (480 kg / 16 TB).

Reported end point At 90 $kgCO_2e/TB$, GreenCache's saving versus the full cache reaches up to 25% in this sensitivity. Heavier SSD carbon makes the adaptive shrink more valuable.

What this does not measure The bars are the accounting model under a changed emission factor. They are not a second hardware campaign.

Source: GreenCache, Sec. 6.6.3, Fig. 20, Table 1

Back to Main 19 Next

Discussion Index

Limits, and What We Would Not Claim

Trace Section 7: the public Azure LLM trace covers about one week. A stronger load model would need a longer trace. The authors treat the predictor as replaceable.

Embodied carbon SSD is charged by allocated capacity and time under amortized accounting. Releasing capacity does not erase manufacturing emissions; the paper assumes others can use that capacity.

MoE, from the authors' discussion Mixture-of-experts models can cut activated compute while leaving KV storage in place, which would make the SSD term relatively larger. The experiments themselves are dense Llama-3, not MoE runs.

Future work, ours The same capacity question could be asked across data centers in different regions, and HBM–HBF could be treated as cache tiers. Nothing in this paper measures multi-region placement, HBM, or HBF, and the 15.1% result does not transfer to that hardware.

Eviction boundary Discarding SSD KV entries does not move all KV back into GPU HBM. Solver sets capacity; LCS selects replacements.

Source: GreenCache, Secs. 5.5, 7. The HBF sentence is our research note.

Back to Main 19

Discussion Index

Discrete Capacity & Solver Assumptions

Allocation granularity SSD capacity is chosen in 1 TB steps. Section 5.4.2 keeps S_t on a discrete integer set, so sizes between those steps are excluded. The paper treats the resulting rounding loss as the cost of a small search.

Configured maxima The platform provides up to 16 TB of SSD. The evaluated maximum is 16 TB for Llama-3 70B and 8 TB for Llama-3 8B.

Decision interval The solver chooses the cache size again every hour. Inside that interval, CI_t is held constant because the CI dataset is hourly.

Profile-based feasibility SLO checks use offline profiles taken after cache warm-up with the LCS replacement policy. Cache size, hit rate, and latency stay implicit in those profiles.

What can move the choice Load prediction, CI prediction, and profiling error can change the selected capacity relative to a ground-truth run. Section 6.5 measures that effect on carbon savings.

Why the search stays small Section A reduces the general form to 0–1 Knapsack, so the unrestricted problem is NP-hard. This instance stays small because the candidates are the 1 TB grid over a finite hourly horizon.

Source: GreenCache, Secs. 5.4.2–5.4.3, 6.1, 6.5; Sec. A

Back to Main 7 Main 8 Discussion Index

Detailed Document QA LCS Example

$$\text{Score}_{\text{doc}} = \frac{\#\text{Hit} \times \text{AccuDocLen}}{\text{Size} \times \text{Age}} \quad \text{Eq. (9)}$$

#Hit: Document reuse events across questions, not a token count.
AccuDocLen: Total reused document tokens, not the SSD footprint.
Size / Age: Occupied SSD space / time stored; both penalize the score.

Hypothetical documents — illustrative values, not paper measurements

Document	#Hit	AccuDocLen	Size (GiB)	Age (h)	Score
A	2	1000	1	2	$2 \times 1000/(1 \times 2) = 1000$
B	4	1000	1	2	$4 \times 1000/(1 \times 2) = 2000$
C	4	3000	2	2	$4 \times 3000/(2 \times 2) = 3000$

Evict A first: Lowest score. More hits (B) or more reused content (C) can justify retention despite a larger footprint.
Access skew: TriviaQA $\alpha = 0.7$ concentrates more questions on popular documents than $\alpha = 0.4$. Both use Eq. (9); these toy entries do not simulate either distribution.
The score is a heuristic, not measured CO_2 or a token-level hit-rate percentage.

Source: GreenCache, Sec. 5.5, Eq. (9); Sec. 6.1. Numerical example is illustrative.

Back to Main 11 | General LCS | Multi-turn | Policies | Access Skew Details | Discussion Index

How Is the TriviaQA Workload Generated?

TriviaQA: document question answering

A QA dataset used to model document comprehension. Multiple requests can ask about the same document.



Reuse requires a matching, retained document prefix. New question tokens still need prefill.

A cache hit is not guaranteed.

Zipf: unequal document popularity

Original document visits are nearly uniform. The authors add Zipf skew to concentrate requests on popular documents.

Top 10% of documents: share of prompts



Paper's approximate access shares (Sec. 6.1).

Higher $\alpha \rightarrow$ more concentrated accesses $\rightarrow$ greater KV reuse opportunities.

Request generation and warm-up

Poisson prompt arrivals; time-varying rates follow a scaled Azure LLM trace. Initialize the document cache with **50k prompts**. Profiling and evaluation use **distinct prompt sets**.

Here α controls document-access skew, not arrival rate, cache size, or carbon intensity.

Source: GreenCache, Secs. 5.5, 6.1, 6.3.2; Eq. (9); Table 3.

Same LCS, different access distribution

Both settings use task-specific **Equation (9)**. Only document access distribution differs. Concentrated reuse can raise token-level hit rates.

Table 3: Llama-3 70B, fixed 4 TB, LCS

$\alpha = 0.4$: **14%** $\alpha = 0.7$: **35%**

Token-level hit rates do not directly give carbon reduction.

Back to Main 12 | Document QA Method | Experimental Settings | LCS Document Example | Full Table 3 | Discussion Index

Full Replacement Policy Comparison — Table 3

Llama-3 70B. Token-level hit rate = reused input tokens / total input tokens.

Size	ShareGPT			TriviaQA $\alpha = 0.4$			TriviaQA $\alpha = 0.7$		
(TB)	FIFO	LRU	LCS	FIFO	LRU	LCS	FIFO	LRU	LCS
1	0.05	0.05	0.08	0.05	0.05	0.06	0.10	0.11	0.19
2	0.11	0.12	0.17	0.08	0.08	0.09	0.16	0.19	0.25
4	**0.19**	**0.21**	**0.28**	**0.12**	**0.12**	**0.14**	**0.23**	**0.26**	**0.35**
8	0.34	0.40	0.47	0.19	0.20	0.22	0.37	0.38	0.44
16	0.52	0.69	0.71	0.31	0.32	0.33	0.47	0.52	0.52

1. Limited Capacity

At 4 TB, LCS improves token-level cache hit rate across all three workloads.
ShareGPT: LRU 0.21 → LCS 0.28. TriviaQA $\alpha = 0.4$: LRU 0.12 → LCS 0.14. TriviaQA $\alpha = 0.7$: LRU 0.26 → LCS 0.35.

2. Abundant Capacity

At 16 TB, the gap becomes smaller for some workloads. Example: TriviaQA $\alpha = 0.7$, LRU = LCS = 0.52.

3. Interpretation

LCS prioritizes entries with higher reuse potential per storage footprint, using age as an additional penalty.
Higher token-level hit rate means more input tokens are reused from cache, potentially avoiding repeated prefill computation.
This table does not directly measure CO_2 savings.

Source: GreenCache, Section 6.3.2, Table 3.

Back to Main 12 | General LCS | Multi-turn | Document QA | TriviaQA α Explained | Discussion Index